

ARCHITECTURE

How FinFlo is built.

An npm-workspace monorepo, an Express 5 + Mongoose 9 server, a React 18 + Vite client, Socket.io real-time, scheduled cron jobs, native Capacitor apps, and a single-service deployment — the engineering behind the platform.

DOCUMENT

Architecture & Tech

AUDIENCE

Engineering · Technical

VERSION

1.0

PRECISION ENGINEERING FOR MODERN FINANCE

MONOREPO

One repository, two apps

FinFlo is an **npm-workspaces monorepo** with two apps: `apps/server` and `apps/client`. In production a single **Express service** serves the built React SPA from the repo-root `/public` directory.

Anything not under `/api`, `/socket.io`, or `/uploads` falls back to `index.html` so React Router can take over client-side routing — the API and the app are served by the same process.



apps/server

Node.js + Express 5 (CommonJS) on Mongoose 9 / MongoDB Atlas — the JSON API and money engine.



apps/client

React 18 + Vite (ES modules + JSX) — the SPA, also packaged as two native mobile apps.

SERVER

The Express server

The server is **Node.js + Express 5** in CommonJS, on **Mongoose 9** against MongoDB Atlas. It is organised so that money logic, controllers, and routing each have a clear home.



Money in services

Financial logic lives in `services/`, not in controllers, where multi-document operations run in transactions.



Barrel-split controllers

Large controllers are split by domain behind thin re-export **barrels**, so routes import the same handlers unchanged.



Central route table

Routes are registered centrally in a `[path, module]` table mounted under `/api`.



Response envelope

Every `/api` response is wrapped in a standard envelope and unwrapped transparently by the client.

CLIENT

The React client

The client is **React 18 + Vite**, written in ES modules and JSX. State is **Jotai**, routing is **React Router 6**, and the UI is built on **Tailwind 3 + Radix** with shadcn-style primitives. HTTP goes through `axios`; user feedback is shown with `sonner` toasts.

Jotai state

Atom-based state, with persisted user and member atoms.

React Router 6

Role-split route modules picked at runtime.

Tailwind + Radix

shadcn-style primitives over Tailwind 3 and Radix UI.

REAL-TIME

Live updates over Socket.io

Socket.io is initialized in the socket handler and attached to requests as `req.io`. It powers chat, notifications, and live branding updates across the app.



Chat

Real-time messaging between participants.



Notifications

Live push of events to connected clients.



Live branding

White-label branding changes propagate instantly.

SCHEDULED JOBS

Cron-driven background work

node-cron jobs live in `services/jobs/{loan,saving,member,payment}Jobs.js`. The `scheduledTasksService` is the registrar — it wires the `cron.schedule(...)` calls. All jobs run in the Asia/Karachi timezone and start once after the DB connects.

SCHEDULED JOB	WHAT IT DOES
Overdue downgrade	Moves overdue loans into the appropriate state.
Late-fee accrual	Accrues late fees on overdue repayments.
Saving-profit accrual / distribution	Accrues and distributes profit on savings.
Default detection	Flags loans that meet default criteria.
Term-deposit maturity	Processes term deposits as they reach maturity.
Scheduled payments	Runs payments queued for a future date.

MOBILE

Two native apps, one codebase

Capacitor 8 packages the same client as **two native apps** — member and business — for Android and iOS. The active app is selected at runtime by `APP_MODE`.



Member app

The customer-facing portal, packaged natively for Android & iOS.



Business app

The staff / admin console, from the same codebase, selected by `APP_MODE`.

DEPLOYMENT

Shipping FinFlo

In production, a **single Express service** serves the SPA. For containerized setups, Docker / docker-compose build separate **server** (`:5001`) and **client** (nginx) images. **Vercel** and **Railway** are deploy targets. Configuration comes from `apps/server/.env`, and startup **hard-fails** on a missing `JWT_SECRET` or `MONGO_URI`.



Single-service prod

One Express process serves both the JSON API and the built React SPA.



Docker images

docker-compose builds a server image (:5001) and an nginx client image separately.



Vercel & Railway

Supported deploy targets for the platform.



Fail-closed config

Startup aborts if `JWT_SECRET` or `MONGO_URI` are missing or weak.

TESTING

Tested in both workspaces

Both workspaces use **Vitest**. The server suite runs against a **real in-memory MongoDB replica set** — so transactions execute for real, not mocked. The client suite runs in **jsdom** with **Testing Library**.



Server — real DB

An in-memory replica set means sessions and transactions run exactly as in production.



Client — jsdom

Component and hook tests run under jsdom with Testing Library.

REFERENCE

The technology stack

LAYER	TECHNOLOGY
Repository	npm workspaces monorepo (apps/server + apps/client)
Server runtime	Node.js + Express 5 (CommonJS)
Data layer	Mongoose 9 on MongoDB Atlas
Client framework	React 18 + Vite (ES modules + JSX)
Client state	Jotai
Client routing	React Router 6
UI	Tailwind CSS 3 + Radix UI (shadcn-style primitives)
HTTP & feedback	axios + sonner toasts
Real-time	Socket.io (attached as req.io)
Scheduled jobs	node-cron (Asia/Karachi)
Mobile	Capacitor 8 (Android + iOS), member & business apps
Deployment	Single Express service · Docker / docker-compose · Vercel · Railway
Testing	Vitest — server on an in-memory replica set, client on jsdom + Testing Library

2

APPS IN THE MONOREPO

6

SCHEDULED JOB GROUPS

2

NATIVE MOBILE APPS

In one sentence

A single Express 5 service serves a React 18 SPA from one npm-workspace monorepo — backed by Mongoose 9 on Atlas, real-time over Socket.io, cron jobs in Asia/Karachi, two native Capacitor apps, and Vitest suites that run transactions for real.